Authentication Tokens (Access & Refresh) Stored in localStorage – Account Takeover via XSS

Severity: **Critical ~ High**

CVSS Score: **9.3 (Critical)**

(This can drop to ~6.1. The score depends on whether token theft requires an XSS injection point.)

CWE

- **CWE-522**: Insufficiently Protected Credentials
- **CWE-539**: Use of Persistent Cookies Containing Sensitive Information
- **CWE-79**: Improper Neutralization of Input During Web Page Generation (XSS - the delivery vector)

Affected Asset

- **URL**: https://crm-stage.crusource.com
- **Parameter/Storage Key**: access_token, refresh_token
- **Environment**: Staging
- **Date Discovered**: 10-10-2026

Why this is a vulnerability:

- localStorage has no *HttpOnly* equivalent so JavaScript can always read it.
- Tokens persist across browser restarts (unlike sessionStorage).
- The presence of a **refresh token** means a single theft grants **long-term access**, even after the short-lived access token expires.
- If the refresh token is not rotated or bound to a device/IP, the attacker can silently renew access indefinitely.

Delivery vector (how an attacker would steal it):
Any of the following would trigger theft:

- Stored XSS in a CRM field (e.g., a customer note, ticket description, or user profile field) that is rendered without encoding.
- Reflected XSS via a URL parameter
- .DOM-based XSS via innerHTML, eval, or unsafe postMessage handling.
- A compromised third-party dependency (npm supply chain, external script tag).
- A malicious browser extension with tabs or <all_urls> permission.

Remediation recommendation:

1. **Move tokens out of** localStorage**.** Store the session in a HttpOnly**,** Secure**,** SameSite=Strict cookie set by the server. JavaScript cannot read HttpOnly cookies, so XSS can no longer steal the session token.
2. **If a SPA architecture requires bearer tokens**, keep the access token **in memory only** (a JS variable), never in localStorage/sessionStorage**.** Use a HttpOnly cookie for the refresh token, and only expose a /refresh endpoint that mints short-lived access tokens.
3. **Rotate refresh tokens** on every use and **detect reuse** (if an old refresh token is presented, invalidate the entire session family).
3. **Bind refresh tokens** to a device fingerprint
4. **Shorten access token lifetime** to 5–15 minutes.
5. **Invalidate all sessions** on password change, email change, and MFA change.

Summary

The CRM application stores long-lived authentication credentials **–** specifically an **access token** and a **refresh token –** in the browser's localStorage. Because localStorage is fully accessible to any JavaScript executing on the origin, any Cross-Site Scripting (XSS) vulnerability, compromised third-party script (e.g., analytics, chat widget, ad SDK), or malicious browser extension can read these tokens, exfiltrate them to an attacker-controlled server, and fully impersonate the victim user **–** including refreshing the session indefinitely using the refresh token.